

Đánh giá công bằng các phương pháp Dynamic BO

Hướng dẫn sửa code và thử nghiệm cho Khánh và Nghi • 19/09/2026

Code base: [njkhanh39/wdbo_reproduce](#) (commit 470825b ngày 16/09/2026).

Cần sửa bộ chạy chung trước khi so MI và Dual Gate với W-DBO. Mục tiêu là xác định: **xóa có ích khi nào, tiêu chí nào chọn điểm tốt hơn, và lợi ích có còn khi tính đủ thời gian chạy không.**

1 W-DBO giải quyết gì và còn thiếu bằng chứng nào

Đóng góp: W-DBO dùng Wasserstein để ước lượng việc bỏ một quan sát làm thay đổi dự đoán tương lai bao nhiêu, rồi quyết định có xóa hay không. Mục tiêu là giảm chi phí GP khi dữ liệu tăng, đồng thời giữ thông tin hữu ích. Điều này chưa chứng minh rằng xóa luôn tốt hơn giữ, hoặc Wasserstein là tiêu chí tốt nhất. [P, mục 3–4, công thức 8–12]

Benchmark: paper dùng 10 hàm tổng hợp và 2 bài gần thực tế, mỗi lần chạy 600 giây, lặp 10 lần. Rastrigin, Schwefel và Styblinski–Tang được đổi thành dạng $g(x) + h(t)$: độ cao thay đổi nhưng vị trí tối ưu đứng yên. Chúng chưa kiểm tra khả năng bám theo cực trị di chuyển. Temperature dùng nội suy dữ liệu cảm biến; WLAN dùng mô phỏng dung lượng mạng. Bộ này là mốc tái lập hữu ích, chưa đủ đại diện cho mọi tình huống thực tế. [P, mục 5; H.1–H.2]

Đối chứng còn thiếu trong paper: GP-UCB giữ dữ liệu và vẫn chịu chi phí thời gian, nhưng không mô hình hóa thời gian như W-DBO. Cần so **chính W-DBO với bước xóa bị tắt** để biết lợi ích nào đến từ xóa. Code chung đã có nhánh này qua `criterion="none"`; giữ lại làm đối chứng bắt buộc. [P, Hình 3, Bảng 2; C, `run_once`]

Giả thuyết cần thử: Wasserstein toàn cục có thể giữ nhiều dữ liệu về một vùng luôn kém tối ưu chỉ vì dự đoán ở đó thay đổi mạnh. Đồng thời, cần thử trường hợp vùng đang bị đánh giá thấp về sau trở thành tốt nhất, để tránh phương pháp mới xóa mất thông tin cần thiết.

2 Chấm chất lượng và đo tốc độ như thế nào

Với bài toán tìm cực đại, **regret là phần giá trị bị mất so với lựa chọn tốt nhất tại cùng thời điểm**. Trong mô phỏng, dùng hàm gốc f để chấm; thuật toán chỉ nhận phép đo có nhiễu $y = f + \epsilon$. Ví dụ, giá trị thật là 10 nhưng đo được 12 thì chấm theo 10, để nhiễu may mắn không được tính thành thành tích.

Regret trung bình theo thời gian phù hợp khi hệ thống phải vận hành liên tục:

$$\bar{R}_{time} = \frac{1}{H} \int_0^H \left[f^*(t) - f(x_{held}(t), t) \right] dt.$$

H là thời lượng chạy; $f^*(t)$ là giá trị tốt nhất lúc t ; $x_{held}(t)$ là cấu hình đang thực sự dùng, kể cả khi đang tính hoặc chờ đo. Chốt trước cấu hình ban đầu và lúc thay cấu hình. Nếu điểm thử chính là cấu hình vận hành, phải chấm cả thời gian thử đó.

Regret trung bình theo query lấy trung bình regret tại các lần gọi hàm. Nó cho biết các điểm được chọn tốt đến đâu, nhưng không trực tiếp tính quãng chờ giữa hai lần gọi. Gọi hàm thừa có thể bỏ qua cả một giai đoạn vận hành kém. Vì vậy dùng regret theo thời gian để xếp hạng hệ thống; regret theo query giúp giải thích kết quả.

Tốc độ: ghi khoảng cách giữa hai lần bắt đầu gọi hàm liên tiếp; báo trung vị, p95 và tổng số query. Ghi thêm thời gian chọn điểm, cập nhật GP, tính tiêu chí xóa, xóa và đo hàm. Regret theo thời gian phản ánh hậu quả của chậm trễ; các số đo tốc độ giúp tìm nguyên nhân. Chạy nhanh hơn chưa chắc tốt hơn nếu các điểm được chọn kém hơn.

3 Hai phép so cần làm riêng

Câu hỏi	Cách thử	Chỉ số dùng để kết luận
Cách chọn điểm xóa nào tốt hơn?	Query ở cùng các mốc thời gian mô phỏng; cùng giới hạn số điểm giữ; vượt giới hạn thì xóa một điểm. Chỉ đổi cách chọn điểm xóa.	Regret theo query so chất lượng lựa chọn khi lịch lấy mẫu giống nhau. Báo thêm regret theo thời gian. Chưa kết luận về lợi ích tốc độ.
Phương pháp hoàn chỉnh nào vận hành tốt hơn?	Cùng thời lượng và diễn biến môi trường; tính đủ thời gian chạy. Mỗi phương pháp tự quyết định khi nào xóa và xóa bao nhiêu.	Regret theo thời gian là chính. Regret theo query, tốc độ và số điểm giữ giải thích vì sao thắng hoặc thua.

Ở phép so thứ nhất, vẫn đo thời gian tính nhưng không cho nó làm thay đổi lịch query mô phỏng. Ở phép so thứ hai, mọi chi phí đều làm môi trường tiến về phía trước. Một tiêu chí chọn điểm tốt nhưng tính quá lâu có thể thắng phép so thứ nhất và thua phép so thứ hai.

Nối các phương pháp vào cùng bộ chạy: mở rộng CRITERIA, build_optimizer, criterion_settings và tham số dòng lệnh trong [common.py](#) để có Wasserstein, MI, Dual Gate, không xóa, xóa điểm cũ nhất và xóa ngẫu nhiên. Mọi nhánh dùng chung đồng hồ, log và cách chấm. Giữ cách nhánh none đang bỏ hẳn clean; không thay bằng alpha=0.

Tách hai chế độ trong cấu hình: một chế độ query theo lịch mô phỏng cố định; một chế độ theo thời gian chạy thực. Ở chế độ thứ nhất, bổ sung max_dataset_size để giới hạn dữ liệu; min_dataset_size hiện có chỉ là số điểm tối thiểu phải giữ. Dùng chung GP, chuẩn hóa, lịch học tham số và hệ số UCB. Tách phần chọn điểm xóa khỏi luật cho phép xóa; không ép gate phải xóa rồi gọi đó là Dual Gate đầy đủ.

Ghi đúng tên đối chứng: bộ chạy hiện mặc định min_dataset_size=15, còn W-DBO gốc cho xuống 2 điểm. Giữ cấu hình theo mã tác giả và cấu hình chung thành hai bản có tên rõ ràng. MI còn dùng ngân sách cộng, W-DBO dùng ngân sách nhân; cùng giá trị alpha không có nghĩa cùng mức xóa. So các tiêu chí ở cùng giới hạn dữ liệu trước, rồi mới so các luật xóa hoàn chỉnh. [C; K]

4 Chọn hàm thử để thấy rõ ưu và nhược điểm

Bắt đầu với hàm có một hoặc hai biến đầu vào: dễ vẽ và dễ kiểm tra giá trị tối ưu tại từng thời điểm. Giữ vài hàm trong paper để đối chiếu, rồi thêm các tình huống sau.

Tình huống	Điều muốn biết
Hàm tĩnh hoặc đổi rất chậm	Giữ dữ liệu có tốt hơn xóa không?
Cực trị di chuyển; hai cực trị đổi ngôi	Có bám kịp và chuyển sang vùng tốt hơn không?
Biến động mạnh ở vùng luôn kém tối ưu	Wasserstein có giữ dữ liệu ít giúp cho quyết định không?
Thay đổi đột ngột; trạng thái cũ quay lại	Có thích nghi và có xóa mất dữ liệu sẽ hữu ích trở lại không?

Chạy cả ngắn và dài, giữ nguyên tốc độ biến đổi của môi trường. Báo tổng query và số điểm giữ; 600 giây chưa tự nó cho biết GP đã đủ lớn để xóa có lợi hay chưa. Thử cả hàm đo nhanh và đo chậm: khi việc đo đã rất lâu, giảm chi phí GP có thể ít tác dụng.

Tối ưu tốc độ GP làm sau: nếu bổ sung cập nhật Cholesky và học lại tham số định kỳ, áp dụng cùng chính sách cho cả các nhánh giữ và xóa dữ liệu.

5 Sửa bộ chạy trước khi mở rộng thử nghiệm

Ưu tiên 1 — Đồng hồ trong `run_once` [C, dòng 103–203]. Dùng `time.perf_counter()`. Sau khi chọn `x`, ghi thời điểm gọi hàm thật; truyền thời điểm đo đó vào `evaluate` và `tell`, còn `clean` nhận thời điểm hiện tại khi bắt đầu xóa. Không dùng lại thời điểm trước khi tối ưu query. Lưu riêng thời gian chọn điểm, đo hàm, fit và clean; đổi tên `t_response` hiện tại thành `t_acq_fit`, rồi tính khoảng cách giữa hai query từ log.

Tách tốc độ môi trường khỏi thời lượng chạy: thời gian môi trường = mốc đầu + tốc độ × số giây đã chạy. Tăng thời lượng không được tự làm môi trường chậm lại như `elapsed/duration_seconds` hiện tại; hàm và bảng cực đại phải phủ hết khoảng thời gian chạy. Bản đầu có thể bắt đầu đo sau khi nạp dữ liệu đầu chung, tại thời điểm không sớm hơn quan sát cuối, và báo riêng chi phí khởi tạo. Kiểm tra hạn chót trước query; chỉ chấm đến hết H.

Ưu tiên 2 — Bổ sung `QUERY_FIELDS` và log [C, dòng 47–53, 175–198]. Hiện chưa lưu tọa độ `x` nên không đủ để chấm lại. Thêm `x`, `y`, thời điểm bắt đầu query, nhận kết quả, hoàn tất cập nhật và cấu hình thực sự áp dụng. Sửa cả `save_run` và `load_run` để ghi/đọc `x` nhiều chiều. Bản đầu quy ước: áp dụng `x` mới khi bắt đầu query, giữ `x` cũ trong lúc chọn điểm; ghi cả cấu hình ban đầu. Log phải tái dựng được cấu hình đang dùng ở mọi thời điểm.

Ưu tiên 3 — Sửa cách chấm và vẽ [C, `summarize_seed`, `save_duration_plot`]. Tách thành bước sau khi chạy: lấy lưới thời gian chung, xác định cấu hình đang dùng, tính lại giá trị hàm và cực đại tại từng mốc rồi tích phân. Không dùng `regret × thời lượng vòng lặp` hiện tại làm `regret` theo thời gian; không coi đường trung bình theo query được nội suy là metric này. Chuyển `objective.oracle(...)` ra khỏi vòng chạy để chi phí chấm không ảnh hưởng lịch query. Cập nhật cả `plot.py`, bảng tổng hợp và nhãn đồ thị. Log cũ thiếu `x` hoặc mốc đổi cấu hình phải chạy lại, không đổi nhãn để dùng tiếp.

6 Phân công và điều kiện bắt đầu chạy kết quả chính

Phân công: Khánh sửa đồng hồ, log và nối MI vào hai chế độ ở mục 3; Nghi làm phần chấm sau chạy, kiểm tra độ đúng và nối Dual Gate. Hai bạn kiểm tra chéo trước khi ghép vào code chung. Khánh kiểm tra độ nhạy số mẫu MI; Nghi thử riêng từng gate và cả hai, ghi rõ khi gate từ chối xóa. Chưa dùng các thử nghiệm này để tuyên bố chứng nhận an toàn ngoài giả định đã kiểm tra.

Ba kiểm tra phải qua: (1) Hàm $f(x, t) = -(x - t)^2$, `x` và `t` trong `[0,1]`, giữ `x=0` suốt `[0,1]`: `regret` theo thời gian phải xấp xỉ 1/3 và hội tụ khi tăng mật độ lưới. (2) Với hàm tĩnh và cấu hình luôn tối ưu, thêm thời gian chờ vẫn cho `regret` bằng 0. (3) Nhánh `none` không gọi `clean` hoặc `score`; log và cách chấm vẫn hoạt động khi không có query mới, hoặc vòng cuối vượt H.

Sau đó chạy hàm tĩnh, cực trị di chuyển, rồi các tình huống ở mục 4. Dùng cùng dữ liệu đầu và ít nhất 10 seed; tách bộ chỉnh tham số khỏi bộ test. Đo lần lượt trên cùng máy và số luồng CPU. Bàn giao cấu hình, phiên bản mã/thư viện, log có `x`, hai loại `regret`, số query, số điểm giữ và thời gian từng bước; báo chênh lệch kèm khoảng tin cậy.

Nguồn để tra: [P] [paper arXiv v2](#): Algorithm 1/Hình 3 tr. 8; Bảng 2 tr. 9; H.1 tr. 26–27; H.2 tr. 27–33. [C experiments/common.py](#); [K mi_optimizer.py](#), hàm `clean`. Code gốc: [optimizer.py](#), [model.py](#). Paper chưa đủ rõ về “average regret”; ghi đây là cách đánh giá đã thống nhất của nhóm, không chắc trùng cách chấm Bảng 2.